

使用 ADK、Agent Engine 和 AlloyDB 建構多代理應用程式

來源：<https://codelabs.developers.google.com/multi-agent-app-with-adk?hl=zh-tw>

步驟數：15

在本程式碼研究室中，您將為廚房翻修專案建構多代理應用程式，以便使用 Vertex AI Agent Development Kit 處理翻修提案建立、處理許可和法規遵循要求，以及整合訂單管理功能。

目錄

1. [1. 總覽](#)
2. [2. 事前準備](#)
3. [3. 原型](#)
4. [4. 設定 ADK](#)
5. [5. 專案結構](#)
6. [6. 原始碼](#)
7. [7. 資料庫設定](#)
8. [8. 模型設定](#)
9. [9. 設定環境變數](#)
10. [10. 執行代理程式](#)
11. [11. 結果](#)
12. [12. 部署至 Agent Engine](#)
13. [13. 單行部署選項](#)
14. [14. 清除所用資源](#)
15. [15. 恭喜](#)

1. 總覽

代理是一種自主程式，會與 AI 模型對話，並使用手邊的工具和情境資訊執行目標導向的作業，還能根據事實自主決策！

如果應用程式有多個代理，可視需要自主協同運作，以達成更大的目標，且每個代理都具備獨立知識，並負責特定領域，那麼應用程式就會成為**多代理系統**。

Agent Development Kit (ADK)

[Agent Development Kit \(ADK\)](#) 是彈性十足的模組化框架，可用於開發及部署 AI 代理。ADK 支援將多個不同的代理例項組合成多代理系統 (MAS)，建構出複雜的應用程式。

在 ADK 中，多代理系統是一種應用程式，不同代理 (通常會形成階層) 會協作或協調，以達成更大的目標。以這種方式建構應用程式可帶來顯著優勢，包括提升模組化、專業化、可重複使用性、可維護性，以及使用專用工作流程代理定義結構化控制流程的能力。

多代理系統注意事項

首先，請務必充分瞭解每位代理商的專業領域和原因。「你知道為什麼需要特定子代理程式來處理某件事嗎？」請先解決這個問題。

第二，如何將這些代理程式與根代理程式整合，以便將每個回應傳送至適當位置並解讀。

第三，您可以參閱這份說明文件，瞭解多種代理程式轉送類型。請確認哪一個適合應用程式流程。此外，您還需要哪些各種情境和狀態，才能控制多代理系統的流量控制。

建構項目

我們來建構多代理系統，處理廚房整修事宜。我們將建構含有 3 個代理程式的系統。

1. Renovation Proposal Agent
2. 許可證和法規遵循檢查代理程式
3. 訂單狀態檢查代理程式

裝修提案代理程式，用來生成廚房裝修提案文件。

許可證和法規遵循代理人，負責處理許可證和法規遵循相關工作。

訂單狀態檢查代理程式，可透過在 AlloyDB 中設定的訂單管理資料庫，檢查物料的訂單狀態。

我們會根據需求，讓根代理程式自動調度管理這些代理程式。

需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

步驟 2 · 約 0 分鐘

2. 事前準備

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 按一下這個[連結](#)，啟動 Cloud Shell。如要在 Cloud Shell 終端機 (用於執行雲端指令) 和編輯器 (用於建構專案) 之間切換，請點選 Cloud Shell 中的對應按鈕。
4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 請務必使用 Python 3.9 以上版本
8. 執行下列指令，啟用下列 API：

```
gcloud services enable artifactregistry.googleapis.com \cloudbuild.googleapis.com  
\run.googleapis.com \aiplatform.googleapis.com
```

9. 如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。
-

步驟 3 · 約 0 分鐘

3. 原型

如果您決定為專案採用「**Gemini 2.5 Pro**」模型，可以略過這個步驟。

前往 [Google AI Studio](#)。開始輸入提示。我的提示如下：

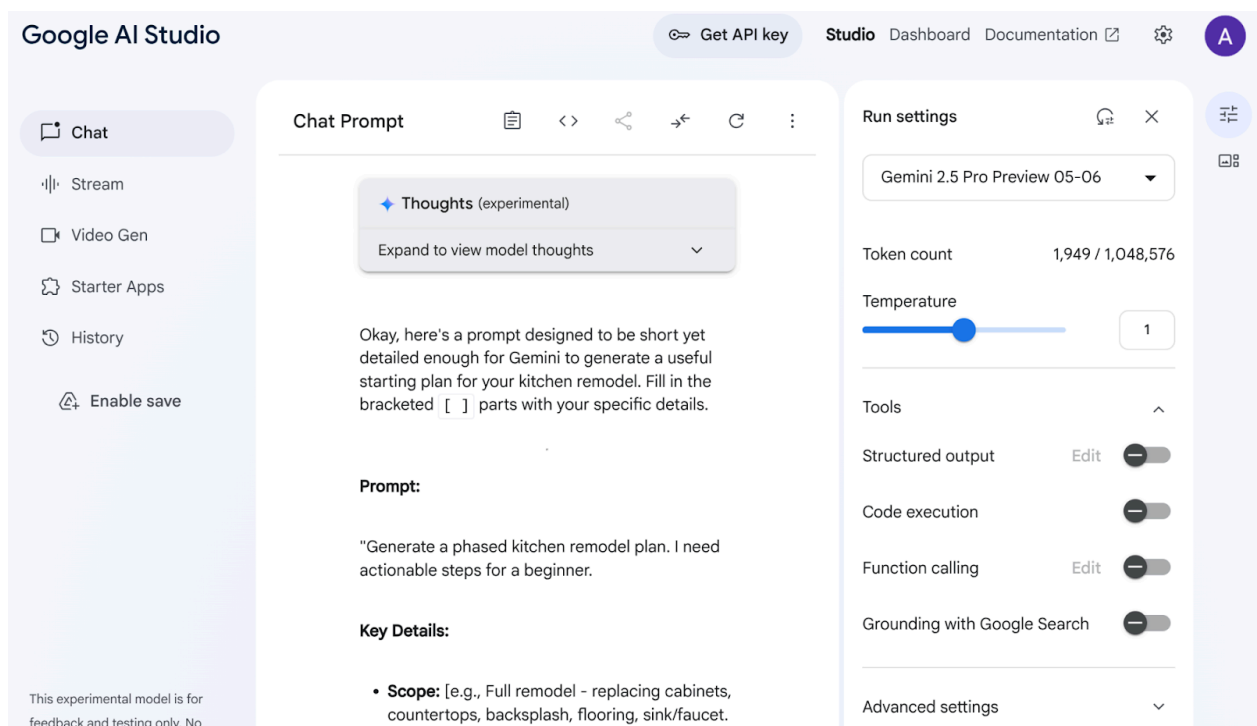
```
I want to renovate my kitchen, basically just remodel it. I don't know where to start. So I want to use Gemini to generate a plan. For that I need a good prompt. Give me a short yet detailed prompt that I can use.
```

調整並設定右側的參數，取得最佳回應。

根據這段簡單的描述，Gemini 為我製作了非常詳細的提示，讓我開始進行裝修！也就是說，我們使用 Gemini，讓 AI Studio 和模型提供更優質的回覆。您也可以根據用途選取要使用的模型。

我們選擇了 Gemini 2.5 Pro。這是「思考型模型」，因此我們能獲得更多輸出詞元，在本例中最多可達 **65,000** 個詞元，適用於長篇分析和詳細文件。啟用 Gemini 2.5 Pro 後，系統會顯示 Gemini 思考方塊。這項模型具備原生推論能力，可接受長脈絡要求。

請參閱下方回應的程式碼片段：



AI Studio 分析我的資料後，生成了櫥櫃、檯面、擋水板、地板、水槽、凝聚力、調色盤和材質選擇等所有項目。Gemini 甚至會引用來源！

反覆選擇不同模型，直到滿意為止。不過，既然有 Gemini 2.5，我認為不必這麼麻煩：)

無論如何，現在請試著使用不同的提示詞，看看這個想法會如何呈現：

```
Add flat and circular light accessories above the island area for my current kitchen in the attached image.
```

附上現有廚房的圖片連結 (或任何[廚房圖片範例](#))。將模型變更為「Gemini 2.0 Flash 圖像生成預先發布版」，即可生成圖像。

我得到以下輸出內容：



這就是 Gemini 的威力！

從解讀影片、生成原生圖片，到以 Google 搜尋為基礎提供真實資訊，這些都是採用 Gemini 技術才能建構的功能。

您可以在 AI Studio 中取得這個原型和 API 金鑰，並運用 Vertex AI ADK 的強大功能，將原型擴展為完整的代理程式應用程式。

步驟 4 · 約 0 分鐘

4. 設定 ADK

1. 建立及啟動虛擬環境 (建議)

在 Cloud Shell 終端機中建立虛擬環境：

```
python -m venv .venv
```

啟動虛擬環境：

```
source .venv/bin/activate
```

2. 安裝 ADK

```
pip install google-adk
```

步驟 5 · 約 0 分鐘

5. 專案結構

3. 在 Cloud Shell 終端機中，於所需專案位置建立目錄

```
mkdir agentic-apps  
cd agentic-apps  
mkdir renovation-agent
```

4. 前往 Cloud Shell 編輯器，然後建立檔案 (一開始為空白)，藉此建立下列專案結構：

```
renovation-agent/  
  __init__.py  
  agent.py  
  .env  
  requirements.txt
```

步驟 6 · 約 0 分鐘

6. 原始碼

5. 前往「`init.py`」，並更新為下列內容：

```
from . import agent
```

6. 前往 `agent.py`，並使用下列路徑中的內容更新檔案：

```
https://github.com/AbiramiSukumaran/adk-renovation-agent/blob/main/agent.py
```

在 `agent.py` 中，我們匯入必要依附元件、從 `.env` 檔案擷取設定參數，並定義 `root_agent`，以協調我們要在這個應用程式中建立的 3 個子代理。我們提供多種工具，協助您處理這些子代理程式的核心和支援功能。

7. 確認您有 Cloud Storage bucket

這是用來儲存代理程式產生的提案文件。請建立這個服務帳戶並授予存取權，讓使用 Vertex AI 建立的多代理系統可以存取。做法如下：

```
https://cloud.google.com/storage/docs/creating-buckets#console
```

將值區命名為「`next-demo-store`」。如果命名為其他名稱，請記得更新 `.env` 檔案中的 `STORAGE_BUCKET` 值 (在「設定環境變數」步驟中)。

8. 如要設定 bucket 的存取權，請前往 Cloud Storage [控制台](#) 和 Storage Bucket (在本例中，bucket 名稱為「next-demo-storage」：<https://console.cloud.google.com/storage/browser/next-demo-storage>)。

依序前往「權限」->「查看主體」->「授予存取權」。將主體選取為「allUsers」，角色選取為「Storage 物件使用者」。

```
Make sure to not enable "prevent public access". Since this is a demo/study application we are going with a public bucket. Remember to configure permission settings appropriately when you are building your application.
```

9. 建立依附元件清單

在 `requirements.txt` 中列出所有依附元件。您可以從 [repo](#) 複製這項資訊。

多代理系統原始碼說明

`agent.py` 檔案使用 Agent Development Kit (ADK) 定義廚房改造多代理系統的結構和行為。以下是主要元件：

代理程式定義

RenovationProposalAgent

這個代理程式負責建立廚房整修提案文件。這項工具可視需要接受輸入參數，例如廚房大小、所需風格、預算和顧客偏好設定。並根據這些資訊，使用大型語言模型 (LLM) Gemini 2.5 生成詳細提案。然後將生成的提案儲存在 Google Cloud Storage bucket 中。

PermitsAndComplianceCheckAgent

這位專員主要負責確保裝修工程符合當地建築法規。這項工具會接收有關提議裝修工程的資訊 (例如結構變更、電氣工程、水管修改)，並使用 LLM 檢查許可證規定和法規遵循規則。代理程式會使用知識庫中的資訊 (您可以自訂知識庫，存取外部 API 來收集相關法規)。

OrderingAgent

這個代理程式 (如果不想立即導入，可以將其註解掉) 會負責檢查裝修所需的材料和設備訂單狀態。如要啟用這項功能，請按照設定步驟建立 Cloud Run 函式。接著，代理程式會呼叫這個 Cloud Run 函式，與含有訂單資訊的 AlloyDB 資料庫互動。這項功能展示如何與資料庫系統整合，追蹤即時資料。

根代理程式 (自動調度管理工具)

root_agent 是多代理系統的中央調度管理工具，這個代理程式會接收初始裝修要求，並根據要求需求判斷要叫用哪些子代理。舉例來說，如果要求需要檢查許可證規定，就會呼叫 PermitsAndComplianceCheckAgent。如果使用者想查看訂單狀態，系統會呼叫 OrderingAgent (如已啟用)。

接著，root_agent 會收集子代理程式的回覆，並合併這些回覆，為使用者提供完整的回覆。這可能包括總結提案、列出必要許可，以及提供訂單狀態更新。

資料流程與重要概念

使用者透過 ADK 介面 (終端機或網頁 UI) 發出要求。

1. 要求會由 root_agent 接收。
2. root_agent 會分析要求，並將要求轉送給適當的子代理。
3. 子代理會使用 LLM、知識庫、API 和資料庫處理要求並生成回覆。
4. 子代理會將回覆傳回給 root_agent。
5. root_agent 會合併回覆，並向使用者提供最終輸出內容。

大型語言模型 (LLM)

這些代理會大量運用 LLM 生成文字、回答問題及執行推理工作。LLM 是代理程式的「大腦」，可瞭解及回應使用者要求。我們在這個應用程式中使用 Gemini 2.5。

Google Cloud Storage

用於儲存產生的裝修提案文件。您必須建立值區，並授予代理程式存取該值區的必要權限。

Cloud Run (選用)

OrderingAgent 會使用 Cloud Run 函式與 AlloyDB 介接。Cloud Run 提供無伺服器環境，可執行程式碼來回應 HTTP 要求。

AlloyDB

如果您使用 OrderingAgent，則需要設定 AlloyDB 資料庫來儲存訂單資訊。我們會在下一節「資料庫設定」中詳細說明。

.env 檔案

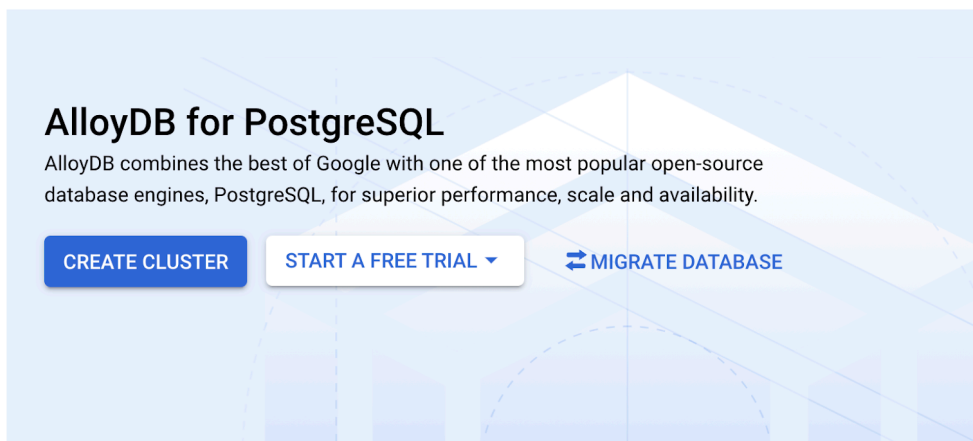
.env 檔案會儲存 API 金鑰、資料庫憑證和 bucket 名稱等機密資訊。請務必妥善保管這個檔案，且不要提交至存放區。此外，也會儲存代理程式和 Google Cloud 雲端專案的設定。root_agent 或支援函式通常會從這個檔案讀取值。請確認 .env 檔案中已正確設定所有必要變數。包括 Cloud Storage bucket 名稱

7. 資料庫設定

在 `ordering_agent` 使用的其中一項工具 (名為「`check_status`」) 中，我們會存取 AlloyDB 訂單資料庫，取得訂單狀態。在本節中，我們將設定 AlloyDB 資料庫叢集和執行個體。

建立叢集和執行個體

1. 在 Cloud 控制台中前往 AlloyDB 頁面。如要在 Cloud 控制台尋找大部分的頁面，只要使用控制台的搜尋列搜尋即可。
2. 選取該頁面中的「建立叢集」：



3. 畫面上會顯示類似下方的內容。使用下列值建立叢集和執行個體 (如果您要從存放區複製應用程式程式碼，請確保值相符)：
- 叢集 ID：「`vector-cluster`」
 - password：「`alloydb`」
 - PostgreSQL 15 / 最新建議版本
 - Region：「`us-central1`」
 - 網路：「`default`」

Configure your cluster

Cluster ID *

Use lowercase letters, numbers, and hyphens. Start with a letter.

Password *



[GENERATE](#)

Set a password for the default "postgres" user. A password is required for the user to log in.

Database version *

PostgreSQL 15 compatible



Location

For better performance, keep your data close to the services that need it. Choice is permanent.

Region *

us-east4 (Northern Virginia)



Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *



[SHOW ADVANCED OPTIONS](#)

4. 選取預設網路後，你會看到如下畫面。

選取「設定連線」。

Networking

Clusters can only be configured with a private IP network path. [Learn more](#)

Network *

default



Private services access connection required

Your network "default" requires a private services access connection. This connection enables your services to communicate exclusively by using internal IP addresses. [Learn more](#)

[SET UP CONNECTION](#)

5. 然後選取「使用系統自動分配的 IP 範圍」並繼續。確認資訊後，選取「建立連結」。

✓ **Enable Service Networking API**

2 **Allocate an IP range**

Google will use this allocated IP range to create subnets.

☐ Select one or more existing IP ranges or create a new one

Select or create an IP range ▼

☒ Use an automatically allocated IP range

Google will automatically allocate an IP range of prefix-length /20 and use the name "default-ip-range".

CONTINUE

3 **Create a connection**

CREATE CONNECTION

CANCEL

6. 設定網路後，即可繼續建立叢集。按一下「CREATE CLUSTER」(建立叢集)，完成叢集設定，如下所示：

Configure your primary instance

Name your instance

Give your instance an ID.

Instance ID *
shopping-cluster-primary

Choice is permanent. Use lowercase letters, numbers, and hyphens. Start with a letter.

Zonal availability

- ☐ Single zone
In case of outage, no failover. Not recommended for production.
- ☒ Multiple zones (Highly available)
Automatic failover to another zone within your selected region. Recommended for production instances. Increases cost.

Machine

AlloyDB's superior performance lets you run your existing workloads on smaller machines.

Machine Type *
8 vCPU, 64 GB

Connectivity

Private IP Connectivity

- ☒ Enable Private IP (always enabled)
Connect from applications and clients within the associated network. [Learn more](#)

CREATE CLUSTER

CANCEL

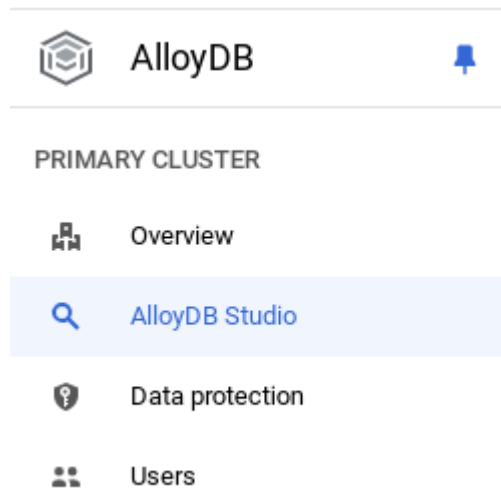
請務必變更執行個體 ID (您可以在設定叢集 / 執行個體時找到)，然後

vector-instance。如果無法變更，請記得在所有後續參照中使用例項 ID。

請注意，建立叢集約需 10 分鐘。成功後，畫面上會顯示您剛建立的叢集總覽。

資料擷取

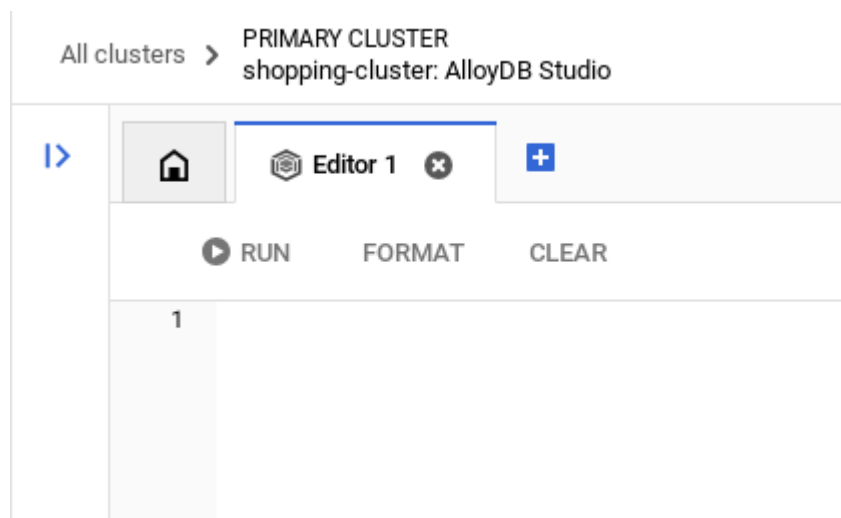
現在要新增包含商店資料的表格。前往 AlloyDB，選取主要叢集，然後選取 AlloyDB Studio：



您可能需要等待執行個體建立完成。完成後，請使用建立叢集時建立的憑證登入 AlloyDB。使用下列資料向 PostgreSQL 進行驗證：

- 使用者名稱：「postgres」
- 資料庫：「postgres」
- 密碼：「alloydb」

成功驗證 AlloyDB Studio 後，即可在編輯器中輸入 SQL 指令。如要新增多個編輯器視窗，請按一下最後一個視窗右側的加號。



您會在編輯器視窗中輸入 AlloyDB 指令，並視需要使用「執行」、「格式化」和「清除」選項。

建立資料表

您可以在 AlloyDB Studio 中使用下列 DDL 陳述式建立資料表：

```
-- Table DDL for Procurement Material Order Status

CREATE TABLE material_order_status (
  order_id VARCHAR(50) PRIMARY KEY,
  material_name VARCHAR(100) NOT NULL,
  supplier_name VARCHAR(100) NOT NULL,
  order_date DATE NOT NULL,
  estimated_delivery_date DATE,
  actual_delivery_date DATE,
  quantity_ordered INT NOT NULL,
  quantity_received INT,
  unit_price DECIMAL(10, 2) NOT NULL,
  total_amount DECIMAL(12, 2),
  order_status VARCHAR(50) NOT NULL, -- e.g., "Ordered", "Shipped", "Delivered",
  "Cancelled"
  delivery_address VARCHAR(255),
  contact_person VARCHAR(100),
  contact_phone VARCHAR(20),
  tracking_number VARCHAR(100),
  notes TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  quality_check_passed BOOLEAN, -- Indicates if the material passed quality control
  quality_check_notes TEXT,      -- Notes from the quality control check
  priority VARCHAR(20),          -- e.g., "High", "Medium", "Low"
  project_id VARCHAR(50),        -- Link to a specific project
  receiver_name VARCHAR(100),    -- Name of the person who received the delivery
  return_reason TEXT,            -- Reason for returning material if applicable
  po_number VARCHAR(50)          -- Purchase order number
);
```

插入記錄

從上述 `database_script.sql` 指令碼複製 `insert` 查詢陳述式，並貼到編輯器。

按一下「執行」。

資料集已準備就緒，現在來建立 Java Cloud Run 函式應用程式，以便擷取狀態。

以 Java 建立 Cloud Run 函式，擷取訂單狀態資訊

1. 從這裡建立 Cloud Run 函式：<https://console.cloud.google.com/run/create?deploymentType=function>
2. 將函式名稱設為「**check-status**」，並選擇「**Java 17**」做為執行階段。
3. 由於這是示範應用程式，您可以將驗證設為「允許未經驗證的叫用」。
4. 選擇 **Java 17** 做為執行階段，並選擇內嵌編輯器做為原始碼。
5. 此時，編輯器會載入預留位置程式碼。

取代預留位置程式碼

1. 將 Java 檔案名稱變更為「**ProposalOrdersTool.java**」，並將類別名稱變更為「**ProposalOrdersTool**」。

2. 將 ProposalOrdersTool.java 和 pom.xml 中的預留位置程式碼，換成這個[存放區](#)「Cloud Run Function」資料夾中相應檔案的程式碼。
3. 在 ProposalOrdersTool.java 中找到下列程式碼行，並將預留位置值替換為您設定的值：

```
String ALLOYDB_INSTANCE_NAME = "projects/<<YOUR_PROJECT_ID>>/locations/us-central1/clusters/<<YOUR_CLUSTER>>/instances/<<YOUR_INSTANCE>>";
```

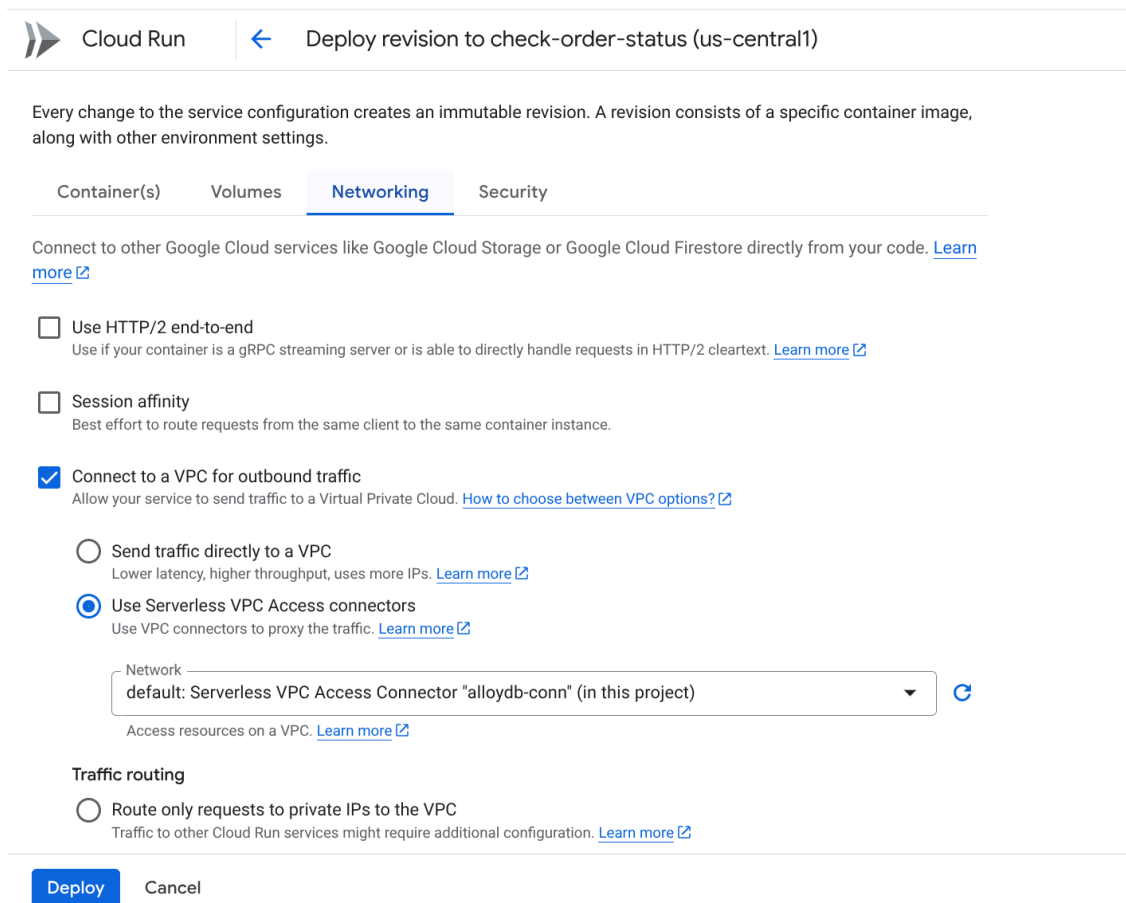
4. 按一下「建立」。
5. 系統會建立並部署 Cloud Run 函式。

重要步驟：

部署完成後，為了允許 Cloud 函式存取 AlloyDB 資料庫執行個體，我們將建立 VPC 連接器。

部署完成後，您應該就能在 Google Cloud Run Functions 控制台中看到函式。搜尋新建立的函式 (check-status)，然後依序點選該函式和「EDIT AND DEPLOY NEW REVISIONS」(編輯並部署新修訂版本) (Cloud Run Functions 控制台頂端的「編輯」圖示 (筆))，並變更下列項目：

1. 前往「網路」分頁：



Cloud Run Deploy revision to check-order-status (us-central1)

Every change to the service configuration creates an immutable revision. A revision consists of a specific container image, along with other environment settings.

Container(s) Volumes **Networking** Security

Connect to other Google Cloud services like Google Cloud Storage or Google Cloud Firestore directly from your code. [Learn more](#)

☐ Use HTTP/2 end-to-end
Use if your container is a gRPC streaming server or is able to directly handle requests in HTTP/2 cleartext. [Learn more](#)

☐ Session affinity
Best effort to route requests from the same client to the same container instance.

☒ Connect to a VPC for outbound traffic
Allow your service to send traffic to a Virtual Private Cloud. [How to choose between VPC options?](#)

☐ Send traffic directly to a VPC
Lower latency, higher throughput, uses more IPs. [Learn more](#)

☒ Use Serverless VPC Access connectors
Use VPC connectors to proxy the traffic. [Learn more](#)

Network
default: Serverless VPC Access Connector "alloydb-conn" (in this project)

Access resources on a VPC. [Learn more](#)

Traffic routing

☐ Route only requests to private IPs to the VPC
Traffic to other Cloud Run services might require additional configuration. [Learn more](#)

Deploy Cancel

2. 選取「連線至虛擬私有雲，以傳出流量」，然後選取「使用無伺服器 VPC 存取連接器」
3. 在「網路」下拉式選單下方，按一下「網路」下拉式選單，然後選取「新增虛擬私有雲連接器」選項 (如果您尚未設定預設連接器)，並按照彈出式對話方塊中的操作說明進行：

Create connector

Name *

Region *

us-central1

A region is a specific geographical location where you can run your resources.

Network *

default

Subnet *

Select an unused /28 subnet or create a new one by entering an unused /28 IP range. The VPC Connector will create connector instances on this subnet.

▼ SHOW SCALING SETTINGS

CREATE

CANCEL

4. 提供 VPC 連接器的名稱，並確認區域與執行個體相同。將「網路」值保留為預設值，並將「子網路」設為「自訂 IP 範圍」，IP 範圍為 10.8.0.0 或類似的可用範圍。
5. 展開「顯示縮放設定」，確認設定完全符合下列條件：

Scaling Settings

Minimum instances *

2

The minimum number of instances provisioned at any time. The connector will automatically scale out if more capacity is needed. Minimum number of instances cannot be decreased later. Larger values increase your cost.

Maximum instances *

3

The maximum number of instances provisioned at any time. The connector will not automatically scale out above this value. Maximum number of instances cannot be decreased later. This setting limits your maximum cost.



Connectors don't scale down automatically or manually. Once the connector has reached maximum number of instances it will remain at this number.

Instance type *

f1-micro

Larger instances support higher bandwidth but raise your costs. [Learn more](#)

^ HIDE SCALING SETTINGS

CREATE

CANCEL

6. 按一下「建立」，這個連接器現在應該會列在輸出設定中。
 7. 選取新建立的連接器。
 8. 選擇透過這個虛擬私有雲連接器轉送所有流量。
 9. 依序點選「NEXT」和「DEPLOY」。
 10. 更新後的 Cloud 函式部署完成後，您應該會看到產生的端點。
 11. 點選 Cloud Run Functions 控制台頂端的「測試」按鈕，然後在 Cloud Shell 終端機中執行產生的指令，即可測試函式。
 12. 部署的端點是您需要在 .env 變數 `CHECK_ORDER_STATUS_ENDPOINT` 中更新的網址。
-

8. 模型設定

代理理解使用者要求和生成回覆的能力，是由大型語言模型 (LLM) 驅動。您的代理程式需要安全地呼叫這個外部 LLM 服務，因此需要驗證憑證。如果沒有有效的驗證，LLM 服務會拒絕代理程式的要求，代理程式也無法運作。

1. 從 [Google AI Studio](#) 取得 API 金鑰。
 2. 在下一個步驟中設定 `.env` 檔案時，請將 `<<your API KEY>>` 換成實際的 API 金鑰值。
-

9. 設定環境變數

1. 在這個 [repo](#) 的範本 .env 檔案中，設定參數的值。以我的情況來說，.env 包含下列變數：

```
GOOGLE_GENAI_USE_VERTEXAI=FALSE
GOOGLE_API_KEY=<<your API KEY>>
GOOGLE_CLOUD_LOCATION=us-central1 <<or your region>>
GOOGLE_CLOUD_PROJECT=<<your project id>>
PROJECT_ID=<<your project id>>
GOOGLE_CLOUD_REGION=us-central1 <<or your region>>
STORAGE_BUCKET=next-demo-store <<or your storage bucket name>>
CHECK_ORDER_STATUS_ENDPOINT=<<YOUR_ENDPOINT_TO_CLOUD
FUNCTION_TO_READ_ORDER_DATA_FROM_ALLOYDB>>
```

將預留位置替換為您的值。

步驟 10 · 約 0 分鐘

10. 執行代理程式

2. 使用終端機前往代理專案的上層目錄：

```
cd renovation-agent
```

3. 安裝所有依附元件

```
pip install -r requirements.txt
```

4. 您可以在 Cloud Shell 終端機中執行下列指令，執行代理程式：

```
adk run .
```

5. 如要在 ADK 佈建的網頁版 UI 中執行，請執行下列指令：

```
adk web
```

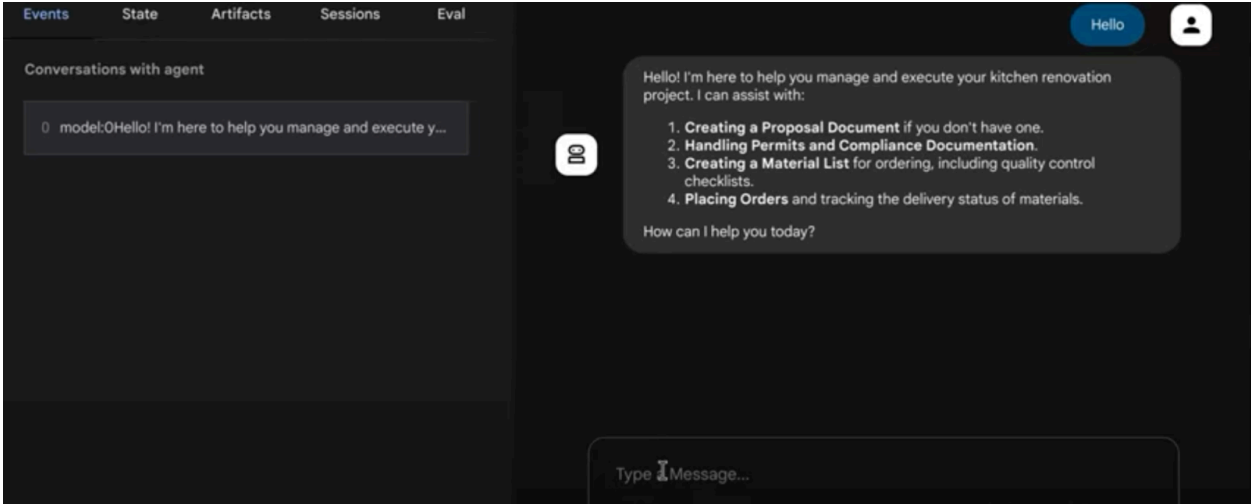
6. 使用下列提示詞進行測試：

```
user>>
```

```
Hello. Generate Proposal Document for the kitchen remodel requirement. I have no other specification.
```

11. 結果

@ [Multi-agent system for Kitchen Renovation tasks](#)



12. 部署至 Agent Engine

您已測試多代理系統，確認運作正常，接下來要將系統設為無伺服器，並在雲端上提供給任何人 / 任何應用程式使用。取消註解 repo 中 [agent.py](#) 的下列程式碼片段，即可部署多代理系統：

```
# Agent Engine Deployment:
# Create a remote app for our multiagent with agent Engine.
# This may take 1-2 minutes to finish.
# Uncomment the below segment when you're ready to deploy.

app = AdkApp(
    agent=root_agent,
    enable_tracing=True,
)

vertexai.init(
    project=PROJECT_ID,
    location=GOOGLE_CLOUD_LOCATION,
    staging_bucket=STAGING_BUCKET,
)

remote_app = agent_engines.create(
    app,
    requirements=[
        "google-cloud-aiplatform[agent_engines,adk]>=1.88",
        "google-adk",
        "pysqlite3-binary",
        "toolbox-langchain==0.1.0",
        "pdfplumber",
        "google-cloud-aiplatform",
        "cloudpickle==3.1.1",
        "pydantic==2.10.6",
        "pytest",
        "overrides",
        "scikit-learn",
        "reportlab",
        "google-auth",
        "google-cloud-storage",
    ],
)

# Deployment to Agent Engine related code ends
```

再次從專案資料夾中執行 `agent.py`，並使用下列指令：

```
>> cd adk-renovation-agent

>> python agent.py
```

這項作業要幾分鐘才能完成。完成後，您會收到類似下列形式的端點：


```
'projects/123456789/locations/us-central1/reasoningEngines/123456'
```

您可以新增「[test.py](#)」檔案，並使用下列程式碼測試已部署的代理：

```
import vertexai
from vertexai.preview import reasoning_engines
from vertexai import agent_engines
import os
import warnings
from dotenv import load_dotenv
load_dotenv()

GOOGLE_CLOUD_PROJECT = os.environ["GOOGLE_CLOUD_PROJECT"]
GOOGLE_CLOUD_LOCATION = os.environ["GOOGLE_CLOUD_LOCATION"]
GOOGLE_API_KEY = os.environ["GOOGLE_API_KEY"]
GOOGLE_GENAI_USE_VERTEXAI=os.environ["GOOGLE_GENAI_USE_VERTEXAI"]
AGENT_NAME = "adk_renovation_agent"
MODEL_NAME = "gemini-2.5-pro-preview-03-25"
warnings.filterwarnings("ignore")
PROJECT_ID = GOOGLE_CLOUD_PROJECT

reasoning_engine_id = "<<YOUR_DEPLOYED_ENGINE_ID>>"

vertexai.init(project=PROJECT_ID, location="us-central1")
agent = agent_engines.get(reasoning_engine_id)
print("*****")
print(agent)
print("*****")

for event in agent.stream_query(
    user_id="test_user",
    message="I want you to check order status.",
):
    print(event)
```

在上述程式碼中，取代預留位置「<<YOUR_DEPLOYED_ENGINE_ID>>」的值，然後執行「`python test.py`」指令，即可執行這項指令，與部署 Agent Engine 的多代理系統互動，並準備好改造廚房！

13. 單行部署選項

您已測試部署的多代理系統，現在讓我們瞭解更簡單的方法，抽象化上一步驟中執行的部署步驟：單行部署選項：

1. Cloud Run：

語法：

```
adk deploy cloud_run \  
--project=<<YOUR_PROJECT_ID>> \  
--region=us-central1 \  
--service_name=<<YOUR_SERVICE_NAME>> \  
--app_name=<<YOUR_APP_NAME>> \  
--with_ui \  
./<<YOUR_AGENT_PROJECT_NAME>>
```

在這種情況下：

```
adk deploy cloud_run \  
--project=<<YOUR_PROJECT_ID>> \  
--region=us-central1 \  
--service_name=renovation-agent \  
--app_name=renovation-app \  
--with_ui \  
./renovation-agent
```

您可以將部署的端點用於下游整合。

2. Agent Engine：

語法：

```
adk deploy agent_engine \  
--project <your-project-id> \  
--region us-central1 \  
--staging_bucket gs://<your-google-cloud-storage-bucket> \  
--trace_to_cloud \  
path/to/agent/folder
```

在這種情況下：

```
adk deploy agent_engine --project <<YOUR_PROJECT_ID>> --region us-central1 --staging_bucket  
gs://<<YOUR_BUCKET_NAME>> --trace_to_cloud renovation-agent
```

您應該會在 Google Cloud 控制台的 [Agent Engine UI](#) 中看到新代理程式。詳情請參閱這篇[網誌](#)。

14. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
-

步驟 15 · 約 0 分鐘

15. 恭喜

恭喜！您已成功使用 ADK 建立第一個代理並與之互動！
